

Library Restoration Task

In this exercise, we are going to look into developing a planning domain using PDDL. Consider an old library with a back room for restoration. The library contains **Standard Books** (sturdy) and **Rare Books** (fragile).

The goal is to move books from the shelves to a workbench, restore them using specific tools, and return them. The actions to solve this problem involve specific constraints:

- **Pick Standard:** You can pick up a standard book freely if your hand is empty.
- **Pick Rare:** Rare books are fragile. You must already be holding a **Dust Jacket** to pick them up. The jacket is wrapped around the book during the process.
- **Pick/Drop Tool:** You can pick up or drop tools (Repair Kits or Dust Jackets).
- **Restore Book:** To restore a book, it must be on the **Workbench**, and you must be holding a **Repair Kit**.

Defining the Domain

Download the package [Library.zip](#) from Canvas.

Your task is to complete the file `library-domain.pddl` to formalize the planning domain. The syntax used in the file is a standardized syntax used in state-of-the-art PDDL solvers, such as in the online editor [planning domains](#).

Note that the file `library-domain.pddl` only contains the definition of the **domain**. The problem instance—including the definition of objects in the world, the initial state, and the goal specification—is given in a separate file. You can find one problem instance for the domain in this exercise in `problem-1.pddl`.

Your task is to complete the code for the actions, which involves writing the **parameters**, the **precondition**, and the **effect**. Note that the actions in the provided file come with comments that give more details than the brief introduction above.

Important Constraints to Implement:

1. **Rare Books:** You cannot pick up a rare book unless you are holding a `jacket`. The effect of picking up the book should remove the jacket from your hand (it is used to wrap the book).
2. **Restoration:** You can only restore a book if the book is at a location marked as a `workbench` and you are holding a `kit`.
3. All the predicates you are allowed to use are already given in the file. You will therefore not need to define any additional requirements or functions.

To verify that your domain definition enables finding a solution to `problem-1.pddl`, you can use the PDDL extension for VSCode or the online solver.

Submission: Please submit your solution to Ex 1.1 as a `.pddl` file named `<LastName>_<FirstName>_domain.pddl`. Make sure that your submitted file does not contain syntax errors.

Designing a Problem

In the previous exercise, you defined the planning domain and tested it on a given problem instance (1 standard book, 1 rare book).

Now the task is to create a new problem file, `problem-2.pddl`, to reflect a more complex scenario. You must define the objects, initial state, and goals based on the description below:

The "Archive" Scenario:

- **Locations:** There are now three locations: `shelf`, `workbench`, and a new location called `archive`.
- **Books:** There are **2 Rare Books** (`rare1`, `rare2`) and **1 Standard Book** (`std1`).
 - All books start at the `shelf`.
- **Tools:**
 - There is **1 Repair Kit** (`kit1`) located at the `workbench`.
 - There are **4 Dust Jackets** (`j1`, `j2`, `j3`, `j4`) located at the `archive`.
- **The Goal:**
 - All three books must be **restored**.
 - The Standard Book (`std1`) must end up at the `archive`.
 - Both Rare Books (`rare1`, `rare2`) must end up back at the `shelf`.

You must create the problem file from scratch (or by copying and modifying the first problem file). Ensure you define all necessary objects and types correctly.

Submission: Please submit your solution to Ex 1.2 as a `.pddl` file named `<LastName>_<FirstName>_problem_2.pddl`.